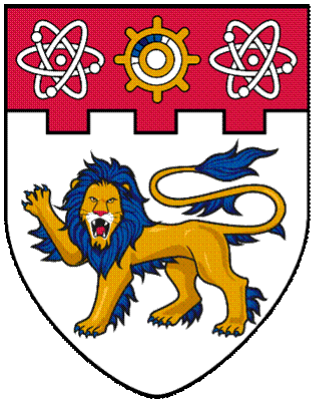


NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SC4064 GPU Programming

Group Project Report

Group: 12

Name	**Matriculation No.**
Tan Jia Xin Annie	U2421334E
Jain Yash	U2222568A
Law Wei Lu	U2223511L

1. Problem Formulation

Attention is the core mechanism of transformer architectures. Given Q, K, V matrices, scaled dot-product attention is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) \cdot V$$

The naive GPU implementation writes the full N×N score matrix QK^T to HBM between each of three kernel launches. At N=4096, d=64, fp16, this is over 1 GB of HBM traffic per forward pass — making the computation **memory-bandwidth bound** rather than compute-bound.

2. Proposed Approach

We adopt **FlashAttention**: an IO-aware formulation that tiles computation into SRAM-sized blocks, avoiding materialisation of the N×N matrix in global memory. We implement this across **12 progressive kernel stages**, each applying one isolated change on an NVIDIA A100-SXM4-40GB.

Benchmark configs:

	d=64	**d=128**
B, nh, B·nh	2, 16, 32	4, 16, 64
Warmup / timed iters	10 / 100	10 / 100
TFLOPS formula	4 × B·nh × N² × d / time	same

3. Stage-by-Stage Optimisations

Stage 0 — Naive 3-Kernel Baseline. Three separate kernels explicitly materialise the full N×N matrix in HBM using scalar FP16 operations. Serial over batch-heads. *(0.82 TFLOPS @ N=4096)*

Stages 1–2 — Kernel Fusion and Tiling. Three kernels are fused into one using online softmax, eliminating HBM materialisation of the score matrix. Stage 2 adds cooperative shared memory tiling (BR=BC=32). Without Tensor Cores, both stages remain below 1 TFLOPS. *(0.50 (Kernel Fusion) → 0.54 (Tiling) TFLOPS)*

Stages 3–4 — Tensor Cores via WMMA. The nvcuda::wmma API replaces scalar FP16 MADs with hardware MMA on 16×16 fragments. Stage 4 doubles the tile to 64×64 with 8 warps. Together: **~17× speedup over Stage 2**. *(8.84 (32x32) → 9.97 (64x64) TFLOPS)*

Stage 5 — FA-2 Deferred Division. Softmax denominator division is moved to a single epilogue, matching the FlashAttention-2 algorithm. No performance change, but enables register-resident O accumulation in later stages. *(9.97 TFLOPS)*

Stage 6 — Async Double-Buffering. cp.async overlaps K/V tile loads with computation via double-buffering. Modest gain — the dominant bottleneck is now shared memory bank conflicts. *(10.45 TFLOPS)*

Stage 7 — Shared Memory Padding. 64-element FP16 rows are exactly 128 bytes — one bank period — causing 8-way bank conflicts on every smem access. Padding rows to 72 elements staggers accesses, giving a **2.8× jump**. *(29.47 TFLOPS)*

Stage 8 — CuTe Abstractions. Kernel rewritten with NVIDIA's CuTe library (CUTLASS 3.x). Swizzle<3,3,3> layouts replace manual padding and guarantee conflict-free access. make_tiled_mma handles TC scheduling. O accumulator kept fully register-resident across the KV loop. *(42.15 TFLOPS)*

Stage 9 — Q Hoisting + Swizzled V^T (CuTe, BR=128). Q fragments are loaded from smem once before the KV loop (saving Tc−1 redundant ldmatrix calls). V is transposed into a swizzled (D, BC) layout to eliminate column-major bank conflicts in GEMM-II. *(61.2 TFLOPS)*

Stage 10 — ldmatrix + 2 Blocks/SM (CuTe). BC=64 keeps smem at 64 KB/block, enabling 2 blocks/SM (16 warps) for MMA latency hiding. __launch_bounds__(256,2) caps registers. Scalar smem→reg copies replaced with warp-cooperative ldmatrix loads. *(70.09 TFLOPS)*

Stage 11 — Custom PTX Library from Scratch. CuTe is abandoned entirely. The full kernel infrastructure (layouts, swizzling, GEMM, softmax) is reimplemented in a bespoke PTX library (stage11_include/), unlocking optimisations the CuTe API cannot express.

Two configurations:

	d=64	**d=128**
BR, BC, warps	128, 64, 4 (128 threads)	128, 64, 4 (128 threads)
smem / block	32 KB → 2 blocks/SM	64 KB → 1 block/SM
Fragment loading	load_0_0_0 (all 8 frags hoisted)	load_2_2_2 (2 frags pipelined)
Result (N=4096)	**146.53 TFLOPS**	**216.00 TFLOPS**
vs official FA-2	**+4.7%**	**+1.6%**

At d=64, load_0_0_0 holds all 8 fragment pairs in registers at once — safe since register pressure is low. At d=128 (16 fragment pairs), this causes severe spilling; load_2_2_2 pipelines 2 fragments at a time, keeping only 8 registers live for K at any moment.

CuTe (Stages 8–10) vs PTX from Scratch (Stage 11):

Aspect	**CuTe**	**PTX from Scratch**
Fragment pipelining	Not available via API	load_2_2_2 double-buffer in gemm.cuh

Softmax	Standard online softmax	First-block O rescale skipped; exp2f-based
Threads/block	256 (8 warps)	128 (4 warps) — less register pressure
Dependencies	CUTLASS 3.x	CUDA + PTX only
Perf (N=4096, d=64)	70.09 TFLOPS	146.53 TFLOPS (**2.1**×)

The 2.1× gain comes from two sources: load_2_2_2 with double-buffering fully feeds the MMA pipeline (vs DefaultCopy's element-at-a-time loads in Stage 10), and halving thread count keeps all accumulators register-resident without spilling at 2 blocks/SM.

4. Related Work

Sparse attention reduces complexity from $O(N^2)$ to $O(Nk)$ by restricting each token to attend to a subset of the sequence [1], but may hurt long-range dependency modelling. **Low-rank attention** approximates the attention matrix through factorisation, reducing complexity to $O(Nr)$ [2], trading accuracy for efficiency. FlashAttention preserves **exact** attention while improving efficiency through IO-aware tiling — no approximation, no dropped interactions.

5. Tools and Libraries

Tool / Library	Role
PTX inline assembly	Direct mma.sync.m16n8k16, ldmatrix, cp.async in Stages 6–11
CuTe (CUTLASS 3.x)	Composable layouts, TiledMMA, TiledCopy, swizzle in Stages 8–10
cuBLAS	Theoretical Tensor Core ceiling (back-to-back GEMMs, no softmax)
PyTorch	Numerical correctness reference

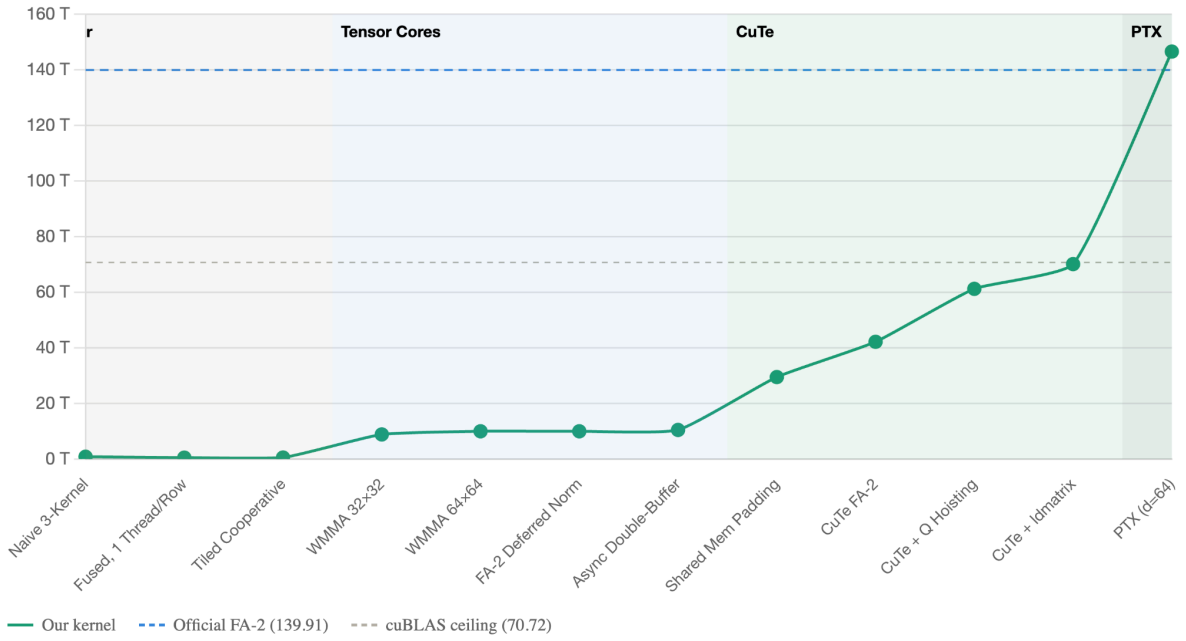
6. Evaluation Results

N=4096, d=64 — Full Stage Results

Stage	Name	Time (ms)	TFLOPS
0	Naive 3-Kernel	168.059	0.82
1	Fused, 1 Thread/Row	272.778	0.50
2	Tiled Cooperative	253.844	0.54
3	WMMA Tensor Cores 32×32	15.552	8.84
4	WMMA 64×64, 8 Warps	13.779	9.97

5	FA-2 Deferred Normalisation	13.779	9.97
6	Async Double-Buffering	13.152	10.45
7	Shared Memory Padding	4.664	29.47
8	CuTe FA-2	3.262	42.15
9	CuTe + Q Hoisting + Swizzled V^T	2.246	61.29
10	CuTe + ldmatrix + 2 Blocks/SM	1.961	70.09
11	**PTX from Scratch (d=64)**	**0.938**	**146.53**
—	Official FlashAttention-2 (d=64)	0.982	139.91
—	cuBLAS ceiling	1.944	70.63

FlashAttention TFLOPS Progression (N=4096, d=64)



Stage 11 exceeds official FA-2 by **4.7%** and exceeds the cuBLAS ceiling by **107%** — possible because FA-2's tiled algorithm reuses K/V from SRAM, achieving higher arithmetic intensity than two independent GEMMs.

N=4096, d=128

	Time (ms)	TFLOPS	vs FA-2
PTX from Scratch (d=128)	2.545	216.00	+1.6%

Official FlashAttention-2	2.587	212.52	—

Key Speedups (N=4096, d=64)

Transition	Speedup	Technique
Stage 0 → 3	10.8×	Tensor Cores
Stage 6 → 7	2.8×	Bank conflict elimination
Stage 10 → 11	2.1×	PTX pipelined fragment loads
Stage 0 → 11	**179×**	Full journey

Team

Name	Contributions
Tan Jia Xin Annie	Stages 9-10. Section 2,4,5 of report
Jain Yash	Stages 0-9, 11. Section 2,4,5 of report
Law Wei Lu	Section 1,3 of report

References

[1] V. Singh, "Demystifying Sparse Attention: A Comprehensive Guide from Scratch," Medium, Jan. 2024. https://medium.com/@vishal09vns/sparse-attention-dad17691478c

[2] F. Alpay, "Low-Rank Attention: Making Large Language Models More Efficient," Medium, Sep. 2025. https://lightcapai.medium.com/low-rank-attention-making-large-language-models-more-efficient-2376b10c2644

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in Advances in Neural Information Processing Systems (NeurIPS), 2022. https://arxiv.org/abs/2205.14135

[4] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in International Conference on Learning Representations (ICLR), 2024. https://arxiv.org/abs/2307.08691

[5] Dao-AILab, flash-attention [Software], GitHub, 2024. https://github.com/Dao-AILab/flash-attention

[6] S. Li, flash_attention_from_scratch, GitHub. https://github.com/sonnyli/flash_attention_from_scratch